



CSCE 633-700: Machine Learning

Neural Networks (Part 1)

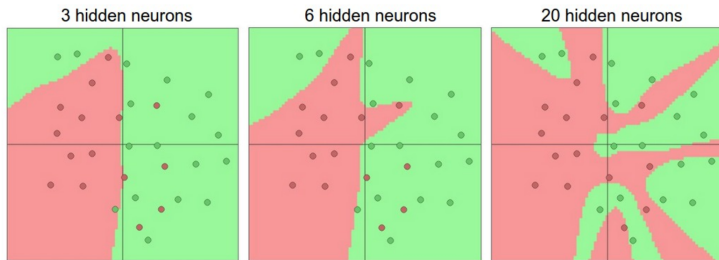
Overview (7)

- Perceptron
 - Representation
 - Learning
 - Examples
- Multilayer Perceptron
 - Representation
 - Learning: Backpropagation
 - Practical issues

Determining number of layers and their sizes (1)

Implementation

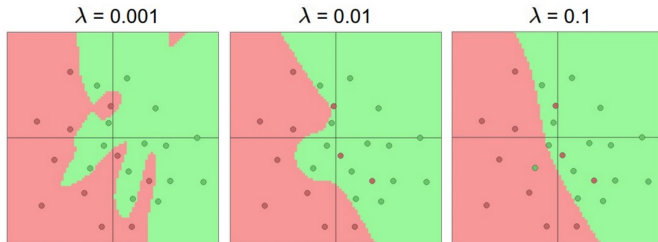
- The capacity of the network (i.e. the number of representable functions) increases as we increase the number of layers
- How to avoid overfitting?



Determining number of layers and their sizes (2)

How to avoid overfitting

- Limit # layers and #hidden units per layers
- Early stopping: start with small weights and stop learning early
- Weight decay: penalize large weights (regularization)
- Noise: add noise to the weights



The effects of regularization strength: Each neural network above has 20 hidden neurons, but changing the regularization strength makes its final decision regions smoother with a higher regularization. You can play with these examples in this [ConvNetsJS demo](http://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html).

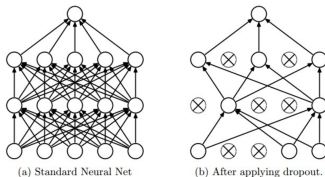
5

<http://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html>

Determining number of layers and their sizes (3)

How to avoid overfitting

- An alternative method that complements the above is **dropout**
- While training, dropout keeps a neuron active with some probability p (a hyperparameter), or sets it to zero otherwise



<https://machinelearningmastery.com/dropout-for-regularizing-deep-neural-networks/>

Determining number of layers and their sizes (4)

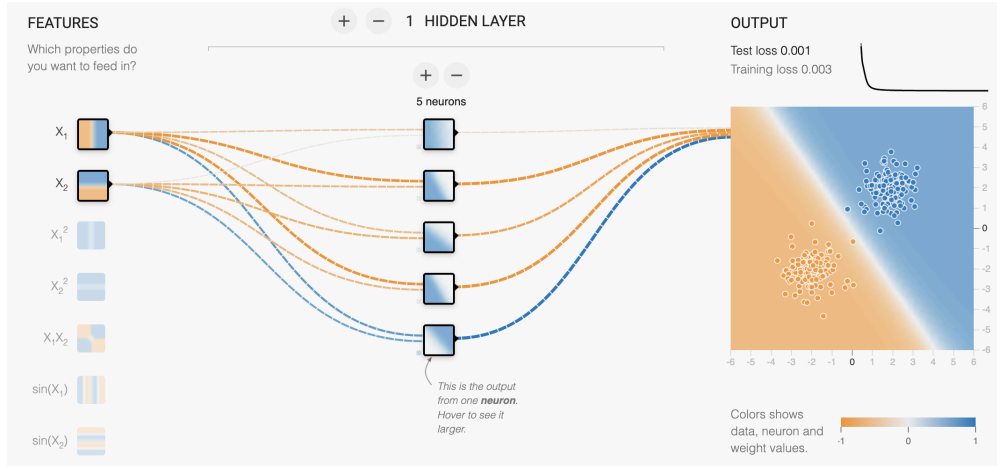
How to chose the number of layers and nodes

- No general rule of thumb, this depends on:
 - Amount of training data available
 - Complexity of the function that is trying to be learned
 - Number of input and output nodes
- If data is linearly separable, you don't need any hidden layers at all
- Start with one layer and hidden nodes proportional to input size
- Gradually increase

Multilayer Perceptron: Practical issues (1)

Linearly separable data

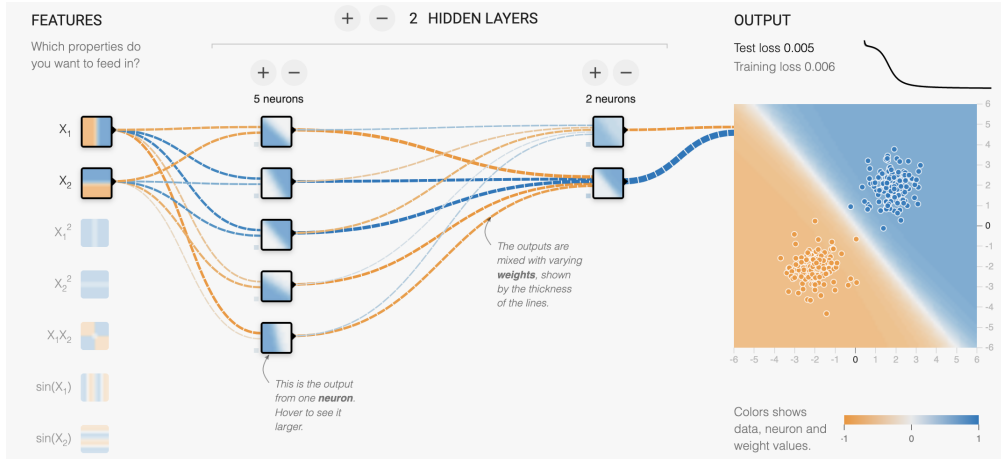
1 hidden layer, learning rate=0.01, epoch=329



Multilayer Perceptron: Practical issues (2)

Linearly separable data

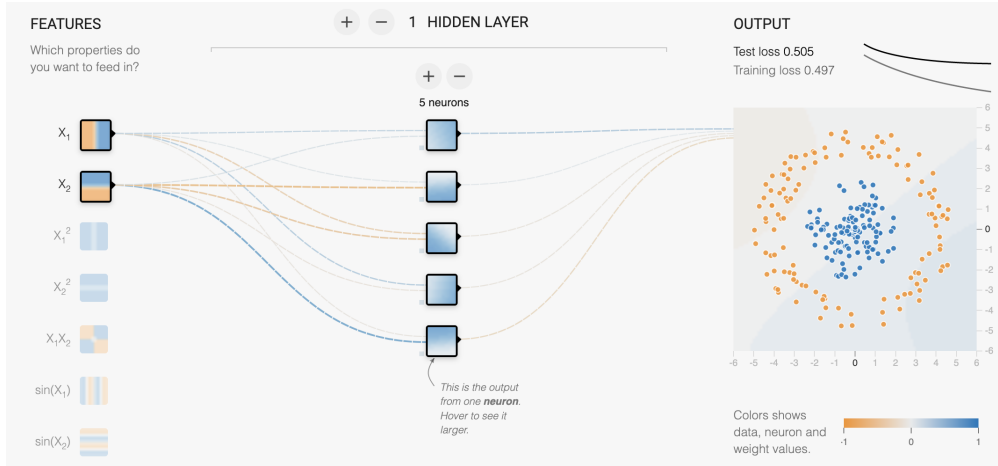
2 hidden layers, learning rate=0.01, epoch=292



Multilayer Perceptron: Practical issues (3)

Non-linearly separable data (spherical shape)

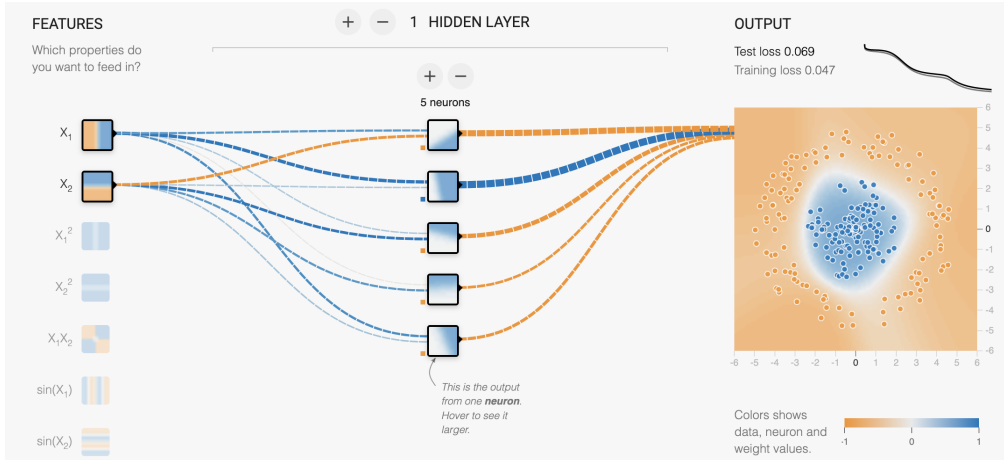
1 hidden layer, learning rate=0.001, epoch=2,028



Multilayer Perceptron: Practical issues (4)

Non-linearly separable data (spherical shape)

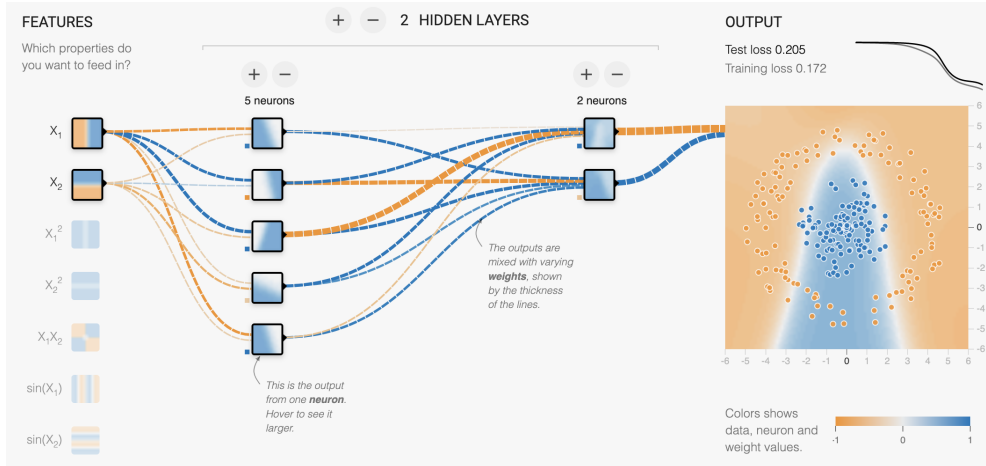
1 hidden layer, learning rate=0.01, epoch=724



Multilayer Perceptron: Practical issues (5)

Non-linearly separable data (spherical shape)

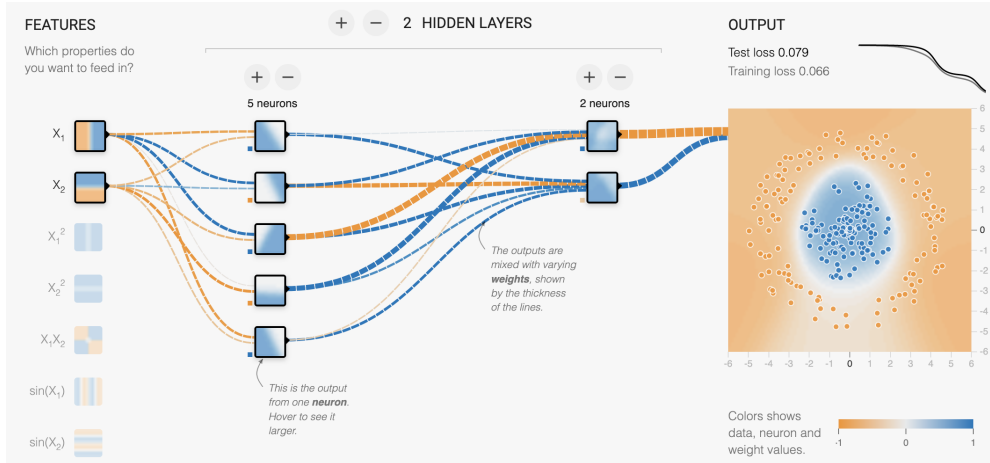
2 hidden layers, learning rate=0.01, epoch=1,062



Multilayer Perceptron: Practical issues (6)

Non-linearly separable data (spherical shape)

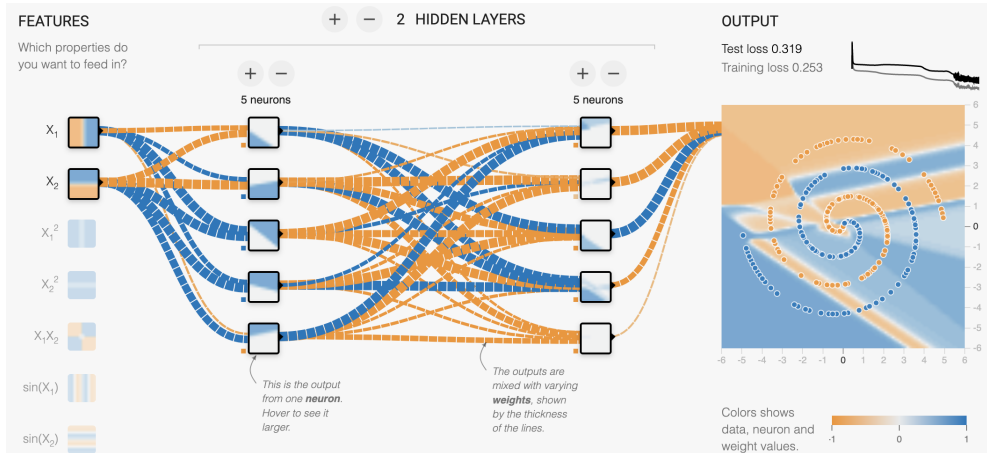
2 hidden layers, learning rate=0.01, epoch=1,210



Multilayer Perceptron: Practical issues (7)

Non-linearly separable data (spiral shape)

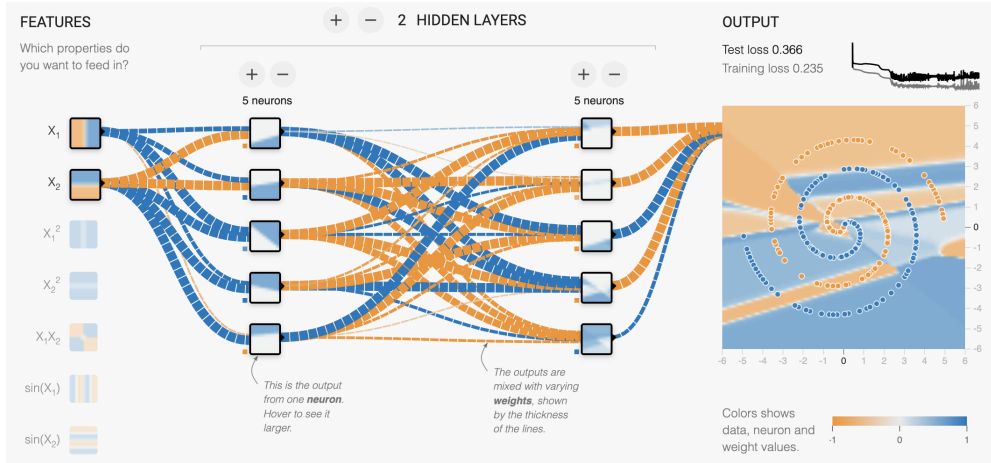
2 hidden layers, learning rate=1, epoch=1,055



Multilayer Perceptron: Practical issues (8)

Non-linearly separable data (spiral shape)

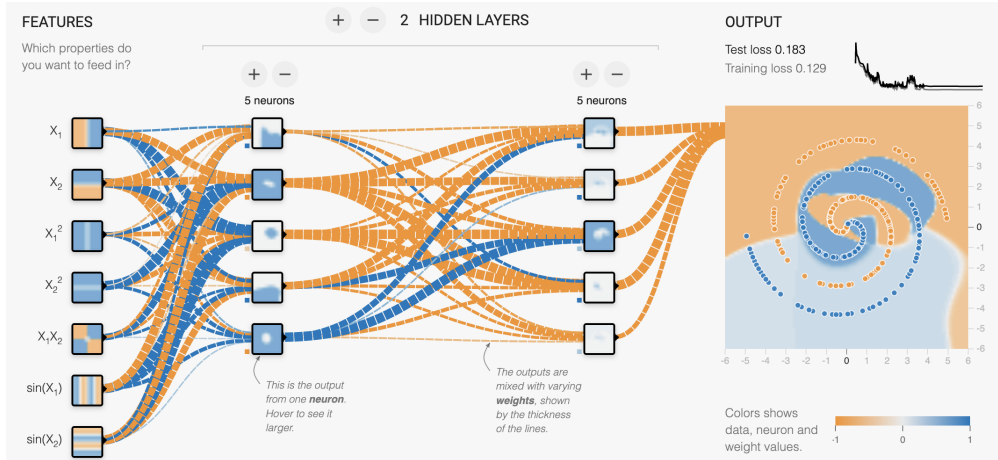
2 hidden layers, learning rate=1, epoch=2,796



Multilayer Perceptron: Practical issues (9)

Non-linearly separable data (spiral shape)

non-linear features, 2 hidden layers, learning rate=1, epoch=402



Multilayer Perceptron: Practical issues (10)

Non-linearly separable data (spiral shape)

non-linear features, 2 hidden layers, learning rate=1, epoch=1,616

